StockManager WinForms

Documento de Especificación Técnica, Requerimientos y Distribución de Roles

C# .NET & SQL SERVER

5	WinForms	SQL	4
Integrantes	Interfaz Gráfica	Base de Datos	Formularios Core

1. Descripción General de la Aplicación

StockManager WinForms es un sistema de escritorio integral desarrollado en C# bajo la plataforma .NET Framework / .NET 8 y respaldado por una base de datos relacional (SQL Server o PostgreSQL). Su objetivo es proporcionar a pequeñas y medianas empresas un control centralizado de inventarios, automatizando la gestión de categorías, productos, actualizaciones de stock y reportes de rendimiento comercial sin requerir conectividad Web o servicios de API.

2. Arquitectura de Navegación y Flujo del Sistema

La aplicación utiliza un esquema MDI (Multiple Document Interface) o formularios modulares. El usuario interactúa con un Menú Principal que invoca pantallas independientes para cada módulo funcional:

> **Flujo Operativo:** [FrmPrincipal] → Invoca → [FrmCategorias] (Gestión de Familias) | [FrmProductos] (Inventario Activo) | [FrmReportes] (Analítica) | [Control Global Excepciones]. Todos los módulos convergen en una clase centralizada de acceso a datos que ejecuta consultas SQL preparadas.

3. Asignación de Roles y Requerimientos Funcionales (RF)

👤 Integrante 1: Arquitectura Base, Conexión y Modelo SQL

Rol: Database & Backend Core Developer Rol 01

- **RF1.1 - Modelado Relacional:** Crear la base de datos `StockManagerDB` con las tablas `Categorias` y `Productos` garantizando llaves primarias autoincrementables e integridad referencial (`FOREIGN KEY`).
- **RF1.2 - Capa de Conexión:** Diseñar la clase `ConexionBD` utilizando ADO.NET o EF Core para proveer métodos de apertura/cierre de conexión leyendo de `app.config`.
- **RF1.3 - Script de Poblado (Seed Data):** Suministrar el archivo `.sql` con datos iniciales para la puesta en marcha de pruebas en el equipo.

👤 Integrante 2: Módulo de Gestión de Categorías

Rol: Frontend & Business Logic (Categorías) Rol 02

- **RF2.1 - Formulario Categorías:** Implementar `FrmCategorias.cs` con controles para consultar, agregar, editar y eliminar registros.
- **RF2.2 - Regla de No Duplicidad:** Impedir en C# el registro de categorías con nombres idénticos existentes en la BD

3. Asignación de Roles y Requerimientos Funcionales (Continuación)

👤 Integrante 3: Módulo de Productos y Control de Stock (Tu Rol)

Rol: Inventory & Business Rules Developer **Rol 03 - TU ROL**

- **RF3.1 - Gestor de Productos (`FrmProductos.cs`):** Diseñar una interfaz interactiva con `DataGridView`, `TextBox` para Nombre/Precio, `NumericUpDown` para Stock y `ComboBox` para Categorías.
- **RF3.2 - Validación de Negocio de Precios:** Verificar mediante código C# que `Precio > 0.00`. Bloquear precios cero o negativos con alerta `MessageBox`.
- **RF3.3 - Carga Dinámica de Categorías:** Alimentar el `ComboBox` de categorías consumiendo la capa del Integrante 2 de manera asíncrona o al cargar el formulario (`Form_Load`).
- **RF3.4 - Control de Entradas/Salidas de Stock:** Crear la función atómica para aumentar o disminuir el inventario, garantizando que el stock resultante jamás sea menor a 0.

👤 Integrante 4: Módulo de Reportes y Exportación

Rol: Data Analytics & File Export Developer **Rol 04**

- **RF4.1 - Reporte de Stock Crítico:** Diseñar en `FrmReportes.cs` una consulta SQL con `INNER JOIN` para listar productos cuya existencia sea inferior o igual a 5 unidades.
- **RF4.2 - Valorización del Inventario:** Calcular mediante `SUM(Precio * Stock)` y `GROUP BY` el capital acumulado total y el valor por categoría.
- **RF4.3 - Exportación a Archivo Plano:** Generar un archivo `.txt` o `.csv` en el disco local con el reporte visualizado en pantalla mediante la clase `StreamWriter`.

👤 Integrante 5: Menú Principal, Integración y Control de Errores

Rol: UX Manager & Quality Assurance **Rol 05**

- **RF5.1 - Dashboard Principal (`FrmPrincipal.cs`):** Crear la ventana contenedor con menú superior (`MenuStrip`) o botones laterales para acceder a todos los formularios del sistema.
- **RF5.2 - Captura Global de Excepciones:** Implementar un bloque de control de errores centralizado para evitar cierres inesperados (`Crash`) ante datos mal formateados por el usuario.
- **RF5.3 - Manual de Instalación y Pruebas:** Consolidar el proyecto final, validar la compilación en conjunto y redactar las instrucciones de despliegue.

4. Diseno Visual y Maquetado de Interfaz (`FrmProductos.cs`)

A continuación se especifica la maqueta visual que debe construir el **Integrante 3** dentro de Visual Studio usando el diseñador de Windows Forms:



5. Estructura de Datos y Datos de Prueba (Seed Data)

Tabla de referencia que debe crearse en SQL y popularse durante el desarrollo:

ID	NOMBRE PRODUCTO	PRECIO ($)	STOCK	CAT. ID	ESTADO DE PRUEBA
1	Laptop HP Pavilion	2500.00	8	1 (Tech)	Operativo Normal
2	Mouse Inalámbrico Logitech	25.50	3	1 (Tech)	**Stock Bajo (≤ 5)**
3	Licuadora Oster 3v	85.00	15	2 (Hogar)	Operativo Normal
4	Set de Ollas Acero	120.00	4	2 (Hogar)	**Stock Bajo (≤ 5)**
5	Cuaderno Argollado 100h	3.50	50	3 (Papelería)	Alto Volumen

> **Nota de Integración:** Al iniciar el desarrollo de los formularios, el Integrante 3 debe consumir la lista de categorías mediante la consulta `SELECT Id, Nombre FROM Categorias` para garantizar sincronización total con el trabajo del Integrante 2.